Viva question for model lab

Exercise 1: Basic Arithmetic and Logic Programs in Assembly

1. **What is assembly language?** Assembly language is a low-level programming language that uses mnemonics to represent machine instructions.

2. **What is an assembler?** An assembler converts assembly-language instructions into machine code.

3. **What is a register?** A register is a small, high-speed storage location inside the CPU.

4. **What is the function of the MOV instruction?** The MOV instruction transfers data between registers, memory, or an immediate operand, depending on the ISA and instruction form.

5. **What is the function of the ADD instruction?** The ADD instruction adds two operands and stores the result in the specified destination.

6. What is the function of the SUB instruction? The SUB instruction subtracts one operand from another and stores the result.

7. What is the difference between arithmetic and logical instructions? Arithmetic instructions perform mathematical operations, whereas logical instructions perform bitwise operations such as AND and OR.

8. What is the function of the AND instruction? The AND instruction produces 1 in a bit position only when both corresponding input bits are 1.

9. What is the function of the OR instruction? The OR instruction produces 1 in a bit position when at least one corresponding input bit is 1.

10. What is the function of the XOR instruction? XOR produces 1 when the corresponding input bits differ and 0 when they are the same.

11. What is an opcode? An opcode is the part of a machine instruction that specifies the operation to be performed.

12. What is a flag register? A flag register stores status information about processor operations, such as zero, carry, sign, and overflow.

13. What is the Zero Flag? The Zero Flag is set when an operation produces a zero result, according to the instruction's flag-setting behavior.

14. What is the difference between ARM and x86 assembly? ARM and x86 use different instruction sets, registers, and instruction syntax. An assembly program must follow the syntax and rules of its target architecture.

15. How do you verify an assembly program's output? Execute the program using a suitable assembler and simulator or processor, then compare the output or register values with the expected result.

Exercise 2: Implementing Addressing Modes Using Assembly

1. What is an addressing mode? An addressing mode specifies how an instruction obtains its operand or calculates the operand's effective address.

2. What is immediate addressing? The operand value is included directly in the instruction.

3. What is register addressing? The operand is stored in a processor register.

4. What is direct addressing? The instruction specifies the memory address where the operand is stored.

5. What is indirect addressing? The instruction uses an address stored in a register or memory location to locate the operand.

6. What is register-indirect addressing? A register contains the memory address of the operand.

7. What is indexed addressing? Indexed addressing uses an index value, often combined with a base address, to calculate the effective address.

8. What is base-plus-offset addressing? The effective address is calculated by adding an offset to a base register value.

9. What is relative addressing? Relative addressing calculates a target address using the program counter and a specified displacement.

10. What is an effective address? An effective address is the calculated memory address used to access an operand.

11. Which addressing mode uses a constant in the instruction? Immediate addressing.

12. Which addressing mode accesses data directly from a register? Register addressing.

13. Why are addressing modes important? They provide flexible ways to access constants, registers, arrays, pointers, and memory locations.

14. What is the difference between immediate and indirect addressing? Immediate addressing supplies the operand value directly, whereas indirect addressing uses a stored address to locate the operand.

15. How can you test addressing modes in an assembly program? Write instructions using different addressing modes, execute them, and inspect the resulting register or memory values.

Exercise 3: Designing a 4-Bit ALU Using Logic Gates in Logisim

1. What is an ALU? An Arithmetic Logic Unit performs arithmetic and logical operations on binary data.

2. What is a 4-bit ALU? A 4-bit ALU operates on two 4-bit operands and produces the result of a selected operation.

3. Which operations are implemented in this exercise? AND, OR, addition, and subtraction.

4. What is a logic gate? A logic gate is a digital circuit that performs a Boolean operation on binary inputs.

5. What is a half adder? A half adder adds two single-bit inputs and produces Sum and Carry outputs.

6. What is a full adder? A full adder adds two input bits and a carry-in bit to produce Sum and Carry-out.

7. How many full adders are needed for a basic 4-bit ripple-carry adder? Four full adders can be used, one for each bit position.

8. What is a carry bit? A carry bit indicates that an addition produces a value beyond the current bit position.

9. How is subtraction performed using 2's complement? Subtraction is performed using A-B= A+B+1

10. What is a multiplexer? A multiplexer selects one of several input signals and sends it to a single output.

11. Why is a multiplexer used in an ALU? It selects the result of the required arithmetic or logical operation.

12. What is a truth table? A truth table lists the output for every possible combination of input values.

13. What are control lines in an ALU? Control lines select the operation that the ALU must perform.

14. What is overflow in a 4-bit signed ALU? Overflow occurs when a signed arithmetic result lies outside the range representable by four bits, which is −8 to +7 in two's complement.

15. What is Logisim? Logisim is a digital logic simulation tool used to design and test circuits such as adders, multiplexers, and ALUs.

Exercise 4: Implementing Booth's Algorithm for Signed Multiplication

1. What is Booth's algorithm? Booth's algorithm is a technique for multiplying signed binary numbers represented in two's complement.

2. Why is Booth's algorithm used? It handles signed multiplication and can reduce the number of arithmetic operations for certain multiplier patterns.

3. What is the role of register A? Register A stores the accumulator or partial product during the multiplication process.

4. What is the role of register Q? Register Q initially stores the multiplier and is updated during the algorithm.

5. What is the role of register M? Register M stores the multiplicand used in addition or subtraction.

6. What is the purpose of Q−1? Q−1 stores the previous least significant bit of the multiplier register for Booth's decision process.

7. **What happens when Q_0Q_{-1}=01?** Add M to A, then perform an arithmetic right shift of the combined registers.

8. **What happens when Q_0Q_{-1}=10?** Subtract M from A, then perform an arithmetic right shift.

9. **What happens when Q_0Q_{-1}=00?** No addition or subtraction is performed; the combined registers are shifted arithmetically to the right.

10. **What happens when Q_0Q_{-1}=11?** No addition or subtraction is performed; the combined registers are shifted arithmetically to the right.

11. What is an arithmetic right shift? It shifts bits right while preserving the sign bit.

12. **How many iterations are required for two n-bit operands?** The basic Booth's algorithm performs n iterations.

13. **How is a negative number represented in Booth's algorithm?** Negative numbers are represented using two's complement.

14. **What is the final product in Booth's algorithm?** The final product is obtained by combining the final A and Q registers.

15. **How can you verify a Booth's algorithm program?** Test positive and negative operands, compare the product with ordinary multiplication, and check boundary cases.

Exercise 5: Simulating Direct-Mapped and Associative Cache Mapping

1. What is cache memory? Cache memory is a small, high-speed memory that stores copies of frequently accessed data and instructions.

2. What is cache mapping? Cache mapping determines where a block of main memory can be placed in the cache.

3. What is direct-mapped cache? Direct-mapped cache assigns each memory block to exactly one cache line.

4. What is fully associative cache? Fully associative cache allows a memory block to occupy any cache line.

5. What is set-associative cache? Set-associative cache allows a memory block to occupy any line within a particular set.

6. What is a cache line? A cache line is a unit of cache storage containing a block of data and its associated metadata.

7. What is a cache hit? A cache hit occurs when the requested data is found in the cache.

8. What is a cache miss? A cache miss occurs when the requested data is not found in the cache.

9. How is a cache hit ratio calculated? Hit ratio equals the number of cache hits divided by the total number of accesses.

10. What is a cache tag? A tag identifies which memory block is stored in a particular cache line.

11. What is a cache index? An index selects a cache line or set.

12. What is a block offset? A block offset identifies a particular byte or word within a cache block.

13. What is a conflict miss? A conflict miss occurs when memory blocks compete for limited cache locations because of mapping restrictions.

14. What is the LRU replacement policy? Least Recently Used replaces the cache line that has not been accessed for the longest time.

15. Why do we simulate cache mapping? Simulation helps us understand cache hits, misses, mapping techniques, replacement policies, and memory performance.

Exercise 6: Demonstrating Memory Access Latency With and Without Cache

1. What is memory access latency? Memory access latency is the time taken to obtain data from a memory location.

2. Why is cache faster than main memory? Cache uses high-speed memory technology and is designed for low-latency access close to the processor.

3. What is temporal locality? Temporal locality means recently accessed data is likely to be accessed again soon.

4. What is spatial locality? Spatial locality means nearby memory locations are likely to be accessed close together in time.

5. What is a cache miss penalty? It is the additional delay required to retrieve data after a cache miss.

6. How can a C program measure memory access time? It can repeatedly access a data structure and measure elapsed time using a suitable timing mechanism.

7. Why should repeated measurements be taken? Repeated measurements help reduce the effect of random timing variations.

8. What is a cache warm-up? Cache warm-up is the process of accessing data before measurement so that relevant data may be loaded into the cache.

9. Why does the first memory access sometimes take longer? The data may not yet be present in the cache, requiring retrieval from a lower memory level.

10. What is sequential memory access? Sequential memory access visits memory locations in consecutive order, such as reading an array from beginning to end.

11. What is random memory access? Random memory access visits memory locations in an unpredictable order, which may reduce the benefits of spatial locality.

12. What is the role of an array in this experiment? An array provides a convenient data structure for comparing sequential or repeated memory-access patterns.

13. Why might cache effects not be clearly visible in a small experiment? Timer resolution, compiler optimization, operating-system activity, and the size of the test data can affect measurements.

14. What is the difference between latency and bandwidth? Latency is the time required for an access, whereas bandwidth is the amount of data transferred per unit time.

15. What conclusion can be drawn from this experiment? Memory hierarchy and cache locality can significantly affect access performance, although measured results depend on the system and test method.

Exercise 7: Simulating a Five-Stage Pipeline

1. What is instruction pipelining? Pipelining overlaps the processing stages of multiple instructions to improve throughput.

2. What are the five stages of a basic pipeline? IF, ID, EX, MEM, and WB.

3. What is the IF stage? It fetches an instruction from instruction memory or the instruction cache.

4. What is the ID stage? It decodes the instruction and identifies the operands.

5. What is the EX stage? It performs arithmetic, logical, address-calculation, or branch-related operations.

6. What is the MEM stage? It accesses data memory for load and store instructions.

7. What is the WB stage? It writes the result into the destination register when required.

8. What is pipeline throughput? Throughput is the number of instructions completed per unit time.

9. What is pipeline latency? Pipeline latency is the time taken for an instruction to pass through the required pipeline stages.

10. What is an ideal five-stage pipeline throughput? In an ideal pipeline, one instruction completes per clock cycle after the pipeline is filled.

11. How many cycles does one instruction require in an ideal five-stage pipeline? It requires five stage cycles to pass through the pipeline, assuming each stage takes one cycle and there are no complications.

12. How many cycles are required for n instructions in an ideal five-stage pipeline? The total is \(n+4\) cycles, assuming one-cycle stages, no hazards, and no stalls.

13. What is a pipeline hazard? A pipeline hazard is a condition that prevents the next instruction from executing in its planned cycle.

14. How can a pipeline be represented in a simulation? A cycle-by-cycle table can show which stage each instruction occupies in every clock cycle.

15. Why is pipeline simulation useful? It helps students understand instruction overlap, throughput, stage timing, and the effects of hazards.

Exercise 8: Identifying and Resolving Pipeline Hazards

1. What is a pipeline hazard? A pipeline hazard prevents an instruction from proceeding in its planned clock cycle.

2. What are the three types of pipeline hazards? Structural hazards, data hazards, and control hazards.

3. What is a structural hazard? It occurs when multiple instructions need the same hardware resource at the same time.

4. What is a data hazard? It occurs when an instruction depends on a result that is not yet available.

5. What is a control hazard? It occurs when a branch or jump affects which instruction should execute next.

6. What is a RAW hazard? Read After Write occurs when an instruction reads a value before an earlier instruction has written it.

7. What is data forwarding? Forwarding supplies a result directly from one pipeline stage to another without waiting for register write-back.

8. What is pipeline stalling? Stalling temporarily delays instruction execution until the required data or resource becomes available.

9. What is a pipeline bubble? A bubble is an unused pipeline slot, often introduced by a stall to prevent an incorrect operation.

10. What is a load-use hazard? It occurs when an instruction immediately uses data loaded by a preceding load instruction before the data becomes available.

11. How does forwarding improve performance? It reduces unnecessary stalls by supplying results directly to dependent instructions.

12. When is forwarding insufficient? Forwarding is insufficient when the required value is not yet available, as can occur with a load-use dependency in a basic pipeline.

13. How can a control hazard be handled? It can be handled using branch prediction, stalling, or flushing incorrectly fetched instructions.

14. What is a pipeline flush? A pipeline flush removes instructions that should not execute, commonly after a mispredicted branch.

15. How do you test hazard resolution? Execute dependent instructions in the simulation and check that register values are correct and the necessary forwarding or stalls occur.

Exercise 9: Simulating Interrupt-Driven I/O

1. What is interrupt-driven I/O? It is an I/O method in which a device interrupts the CPU when it requires service.

2. What is an interrupt? An interrupt is a signal or event that requests processor attention.

3. What is an ISR? An Interrupt Service Routine is the code executed to handle a particular interrupt.

4. What is polling? Polling is a technique in which the CPU repeatedly checks a device's status.

5. How does interrupt-driven I/O differ from polling? Interrupt-driven I/O notifies the CPU when an event occurs, whereas polling requires the CPU to check the device repeatedly.

6. What is an interrupt request? It is a signal or event requesting that the processor handle a device or system event.

7. What is interrupt priority? Interrupt priority determines the order in which competing interrupt requests are serviced.

8. What is an interrupt vector? An interrupt vector identifies the location of the handler associated with an interrupt, according to the system's interrupt mechanism.

9. Why is the processor state saved during an interrupt? Saving the necessary state allows the interrupted program to resume correctly after the handler completes.

10. What is a hardware interrupt? A hardware interrupt is generated by a hardware device or controller to request processor attention.

11. **What is a software interrupt?** A software interrupt is generated by executing a designated instruction or through a software-defined interrupt mechanism.

12. **What is interrupt latency?** Interrupt latency is the time between an interrupt event or request and the start of its handling.

13. **What happens after an ISR completes?** The processor returns to the interrupted program or resumes another appropriate execution context.

14. **Why is interrupt-driven I/O useful?** It allows the CPU to perform other work instead of continuously checking a device, improving efficiency for many I/O workloads.

15. **How can interrupt-driven I/O be simulated in C?** A program can model a device event, set an interrupt flag, invoke a handler function, and record how the event is serviced.

Exercise 10: Designing a Memory-Mapped I/O Interface

1. **What is memory-mapped I/O?** Memory-mapped I/O assigns addresses in the memory address space to hardware device registers.

2. **What is an I/O register?** An I/O register is a hardware register used to exchange data, configure a device, or read its status.

3. **What is an address decoder?** An address decoder examines address bits and selects the appropriate device or register.

4. **What is a control register?** A control register stores configuration values or commands that determine how a device operates.

5. **What is a status register?** A status register contains information about the current state of a device, such as whether data is ready or an operation is complete.

6. **What is a data register?** A data register temporarily stores data being transferred to or from a peripheral.

7. **How does the CPU access a memory-mapped device?** It uses memory-access instructions to read from or write to the addresses assigned to the device registers.

8. **What is the difference between memory-mapped I/O and isolated I/O?** Memory-mapped I/O shares the memory address space with device registers, while isolated I/O uses a separate I/O address space.

9. **What is a read operation in memory-mapped I/O**? A read operation obtains data or status from the selected device register.

10. **What is a write operation in memory-mapped I/O?** A write operation sends data or control information to the selected device register.

11. **What is a chip-select signal?** A chip-select signal enables a particular device or component to respond to an access.

12. **What is the role of control signals in an I/O interface?** Control signals indicate the type and timing of operations, such as read, write, and device selection.

13. **Give an example of a memory-mapped I/O device.** A UART serial controller can expose memory-mapped registers for transmitting data, receiving data, and configuring communication.

14. **What is the advantage of memory-mapped I/O?** It allows device registers to be accessed using ordinary memory-access instructions, providing a consistent programming model.

15. **How can you test a memory-mapped I/O interface design?** Apply different addresses and read/write control signals in a simulator. Verify that the correct register is selected and that data is transferred as expected.